C++ to Assembly with Loops Guidelines and Rubric

CS 410 C++ to Assembly with Loops Guidelines and Rubric

Overview

In this activity, you will again convert a simple application from C++ into assembly code. This time there are loops within the C++ code. The coding for this assignment will be performed in Codio. You will then download the file from Codio for submission, along with the completed C++ to Assembly With Loops Activity Template Word Document. The following resources will help you complete the tasks in this assignment:

- Section 7: How to Generate Assembly from C++ Code in the *Guide to Software Reverse Engineering*
- Section 8: Downloading and Uploading Files in the *Codio Guide*

Prompt

Specifically, you must address the following rubric criteria:

1. Explain the functionality of the C++ code.
    - Use the C++ to Assembly With Loops Activity Template to complete this step.
    - The C++ file is located within the Software Reverse Engineering Playground in the Module Two file folder in Codio. It is also in the following table:

```
#include<iostream>
using namespace std;
int main()
        {
  int num, i;
  int pfoduct=1;
cout<<"Enter a number:\n";
cin>>num;
  for (i=num;i>0; i--)
    product = product * i;
cout<<"The factorial for "<<num << "is: "<< product<< endl;
  return 0;
}
```

2. Convert the C++ file into assembly code.
    - The C++ file can be found in the Software Reverse Engineering Playground in the Module Two file folder in Codio.
3. Align each line of C++ code with the corresponding blocks of assembly code.
    - Use the C++ to Assembly With Loops Activity Template to complete this step.
4. Explain how the blocks of assembly code perform the same tasks as the C++ code.

- Use the C++ to Assembly With Loops Activity Template to complete this step.
- Consider which blocks of assembly code are skeleton code versus actual parts from the C++ program.

What to Submit

C++ to Assembly With Loops Activity Template

This should be a Word document. Use this template to explain the functionality of the lines of C++ code, align the lines of C++ code with the corresponding lines of assembly code, and explain how the assembly code performs the same tasks as the C++ code.

Assembly File (S file)

This file is needed to ensure that the C++ code was successfully converted into assembly code.

AI Usage

If you use gen AI tools to support your work on this assignment, be sure to follow these AI usage guidelines. You must acknowledge your use of these tools in your work. Guidelines on how to cite AI tools can be found in this Shapiro Library guide.

C++ to Assembly With Loops Rubric

Criteria	Exceeds Expectations	Meets Expectations	Partially Meets Expectations	Does Not Meet Expectations	Value
C++ Functionality Explanation	Exceeds expectations in an exceptionally clear, insightful, sophisticated, or creative manner (100%)	Explains the functionality of the C++ code with minimal errors and adequate detail (85%)	Shows progress toward meeting expectations, but with errors or omissions; areas for improvement may include explaining the functionality of the C++ code with minimal errors and richer detail (55%)	Does not attempt criterion (0%)	27
C++ to Assembly Conversion	N/A	Converts C++ file into assembly code (100%)	Shows progress toward meeting expectations, but with errors or omissions; areas for improvement may include converting the C++ file into assembly code (55%)	Does not attempt criterion (0%)	15
Translation Alignment	Exceeds expectations in an exceptionally clear, insightful, sophisticated, or creative manner (100%)	Aligns each line of C++ code with the corresponding blocks of assembly code with minimal errors (85%)	Shows progress toward meeting expectations, but with errors or omissions; areas for improvement may include aligning each line of C++ code with the corresponding blocks of assembly code with fewer errors (55%)	Does not attempt criterion (0%)	22
Assembly Functionality Explanation	Exceeds expectations in an exceptionally clear, insightful, sophisticated, or creative manner (100%)	Explains how the assembly code performs the same tasks as the C++ code with minimal errors and adequate detail (85%)	Shows progress toward meeting expectations, but with errors or omissions; areas for improvement may include explaining how the assembly code performs the same	Does not attempt criterion (0%)	27

			tasks as the C++ code with minimal errors and richer detail (55%)		
Clear Communication	Exceeds expectations with an intentional use of language that promotes a thorough understanding (100%)	Consistently and effectively communicates in an organized way to a specific audience (85%)	Shows progress toward meeting expectations, but communication is inconsistent or ineffective in a way that negatively impacts understanding (55%)	Shows no evidence of consistent, effective, or organized communication (0%)	9
				Total:	100%

Activity Details

- Task: View this topic